

Alexey Mazurenko

AI / LLM ENGINEER

CONTACTS

-  **EMAIL**
lyakoway@gmail.com
-  **GITHUB**
github.com/lyakoway
-  **LINKEDIN**
in/alexey-mazurenko-63068941b
-  **WEBSITE**
lyakoway.vercel.app/contacts
-  **LOCATION**
Moscow (UTC+3) · open to remote and relocation
-  **LANGUAGES**
English — B2 · Russian — native
-  **PHONE**
+7 (977) 270-09-30

HIGHLIGHTS

- ~2,000 docs · ~200 questions/day · Recall@1 87% held-out
- Report preparation: 2 hours → 2 minutes
- ~85% normalized SQL result correctness (held-out)
- Open-source demos — RAG Chat and AI Data Pilot on GitHub

PROFILE

AI / LLM engineer with 7+ years in software engineering, including **2+ years building production LLM systems** at MTS Web Services (MWS AI) — the AI division of MTS, one of Russia's largest telecom operators (~80M+ subscribers). Shipped two internal products end-to-end: a document RAG assistant (~2,000 docs; **held-out Recall@1 53% dense-only → 87% hybrid**) and a multi-agent analytics platform (~85% **normalized SQL result correctness**) that **cut report preparation from 2 hours to 2 minutes**.

Internal production at MTS (~12 teams, ~200 RAG questions/day). Public GitHub demos are independently built personal projects of the same problem class — not MTS source code; production data stays under NDA. RAG production default is **GLM-5.3-flash** (TTFT ~2.5–3 s, cost vs GPT/Claude); OpenAI, Anthropic and Ollama are interchangeable.

Own the path from prototype to production: Python / FastAPI, hybrid retrieval, held-out evaluation, React / Next.js, Docker / Kubernetes / CI/CD.

More about my projects and experience — on my website: lyakoway.vercel.app

EXPERIENCE

AI / LLM Engineer

MTS Web Services (MWS AI)

Apr 2024 — present · Moscow

PROJECTS

RAG Chat — AI assistant with Retrieval-Augmented Generation

An AI assistant for working with documents — uploads PDF, Word and Excel files, answers questions about their content and provides links to the source pages and document fragments.

- **RAG pipeline:** document indexing → chunking (tiktoken) → embeddings (fastembed) → hybrid BM25 + vector retrieval (RRF) → LLM answer generation → source citation.
- **Three modes:** RAG Chat → AI Agent → Vector Search — switching within one application.
- **Architecture:** Python / FastAPI → ChromaDB → fastembed → GLM-5.3-flash default (OpenAI / Anthropic / Ollama) → SSE → React / TypeScript; SQLAlchemy — conversation history.
- **Evaluation:** held-out ~180 queries not used to tune prompts or retrieval. Hybrid BM25 + RRF: Recall@1 53% (dense-only) → 87%. Cross-encoder on fused top-k rejected: RU/EN twins tied, gold chunk dropped (~45 p.p. Recall@1, +~3 s).
- **Quality:** 63 pytest tests + CI; TTFT ~2.5–3 s on GLM-5.3-flash.

AI Data Pilot — multi-agent analytics platform

Automates the path from a user's question to a ready analytical result.

- **Data Agent:** turns natural-language questions into SQL, runs multi-step data analysis, detects trends and deviations, and produces tables, charts and analytical conclusions.
- **Knowledge Agent (RAG):** answers questions over internal technical documentation and uploaded files (PDF, Word, Excel), grounded in the retrieved sources.
- **Architecture:** Python / FastAPI → SQLAlchemy → Agent Loop (ReAct) → Tool Calling → Text-to-SQL → SQL guard → PostgreSQL / ClickHouse → analytics layer → SSE → React / TypeScript; GLM-4.6 default (OpenAI / Anthropic interchangeable); the Knowledge Agent's RAG core — hybrid retrieval BM25 + vector embeddings (fastembed) → LLM → source citation.
- **Quality:** 174 pytest tests + CI; held-out SQL eval — ~98% execution (query ran), ~85% normalized result correctness (bag of values after dropping aliases, ORDER BY and number format — not string exact-match); self-correction on failed SQL (GLM-4.6).

CORE TASKS

- Designed and shipped production AI agents — tool calling, orchestration, error handling and recovery.
- Built and measured RAG pipelines — hybrid BM25 + vector search, source citations; rejected a slower reranker after testing on the held-out set.
- Built held-out evaluation suites — queries unused for prompt/retrieval tuning — for regression of retrieval, prompts and models.
- Implemented Text-to-SQL with SQL guardrails and self-correction.
- Owned AI platform services end-to-end — backend (Python / FastAPI), frontend (React / Next.js), infrastructure (Docker / Kubernetes / CI/CD).

KEY RESULTS

RAG Chat

- Internal MTS production usage (~2,000 documents, ~20k chunks, ~12 teams, ~200 questions/day). Public demo: personal project, not MTS code; prod data under NDA.
- Held-out Recall@1 53% (dense-only) → 87% after hybrid BM25 + RRF; cross-encoder on fused top-k rejected (~45 p.p., +~3 s).
- Cuts lookup from minutes to seconds on regulations, contracts and HR policies.

SKILLS

• LLM & AI Agents

- LLM APIs (OpenAI / Anthropic / GLM-5.3-flash / Ollama)
- AI AgentsAgent Loop (ReAct)
- Tool CallingText-to-SQL
- Multi-agent router (2 specialists)
- Prompt Engineering
- Latency & Cost Tuning (TTFT, \$ per query)

• RAG & Retrieval

- RAGHybrid Search (BM25 + Vector)
- RRFChunking (tiktoken)fastembed
- ChromaDBSource Citations

• Evaluation & Quality

- Held-out eval (no tuning leakage)
- Recall@K / MRRLLM-as-a-Judge
- Grounding / Anti-Hallucination
- Prompt EvaluationRegression Testing

• AI Data & Backend

- PythonFastAPISQLAlchemy
- PostgreSQLClickHouse
- SQL Guard (read-only, timeouts)SSE

• Frontend & Infrastructure

- React / Next.jsTypeScriptDocker
- KubernetesCI/CDGit

EDUCATION

Master's degree — Applied Mathematics
Moscow State University of Civil Engineering (MGSU), 2014

KEY RESULTS

AI Data Pilot

- Internal MTS production usage (~15 analysts, ~80 reporting scenarios/week over PostgreSQL, ClickHouse and Excel). Public demo: personal project, not MTS code; prod databases under NDA.
- Held-out SQL: ~98% execution, ~85% normalized result correctness (bag of values, not exact-match); report prep 2 hours → 2 minutes.
- One interface over PostgreSQL, ClickHouse and Excel, with the SQL dialect adapted automatically.

STACK

RAG Chat

- PythonFastAPISQLAlchemyRAGAI AgentsLLM APIChromaDBfastembed
- OllamaSSEReactTypeScriptVite

AI Data Pilot

- PythonFastAPISQLAlchemyText-to-SQLAgent Loop (ReAct)Tool CallingRAG
- BM25 + Vector SearchfastembedPostgreSQLClickHouseSSEReact 19
- TypeScriptpytest

DEMO

- RAG Chat — case study — [lyakoway.vercel.app/portfolio/rag-chat](#)
- RAG Chat — code — [github.com/lyakoway/ai-RAG-chat](#)
- AI Data Pilot — case study — [lyakoway.vercel.app/portfolio/ai-data-pilot](#)
- AI Data Pilot — code — [github.com/lyakoway/ai-data-pilot](#)

EXPERIENCE

Senior Frontend Developer
MTS Web Services

Feb 2019 — Apr 2024 · Moscow

PROJECTS

MTS Profile

A module for storing and visualizing customer data with access management across ecosystem products.

Ecosystem widgets

An embeddable navigation and personalization module for b2c/b2b products.

KEY RESULTS

- Delivered a profile ownership transfer model and a linked-accounts management model.
- Automated user data verification via Gosuslugi (Russian government services portal) with biometrics.
- Delivered the full access management and authorization cycle — access recovery, sign-in and an authentication-method change history.
- Integrated bank card payments and ecosystem widgets into the company's digital products.

STACK

- ReactNext.jsTypeScriptRedux ToolkitSvelteStyled-ComponentsWebpackJestNode.js
- Express